2. Movie Recommendation Insights

-Scrape movie metadata (ratings, genres, reviews) The dataset used for this project was obtained from kagel, (https://www.kaggle.com/datasets/grouplens/movielens-latest-small).

Description of the dataset The MovieLens Latest Small Dataset, developed by GroupLens Research, is a widely used dataset for studying user behavior and building recommendation systems. It contains around 100,000 ratings from hundreds of users across thousands of movies, making it ideal for exploratory analysis and rapid experimentation. The dataset is organized into key files: ratings.csv (user ratings and timestamps), movies.csv (movie titles and genres), tags.csv (user-generated labels), and links.csv (connections to external databases). Together, these files provide a structured view of how users interact with movies. This dataset enables us to analyze patterns such as rating distributions, user activity levels, and genre preferences. We can identify trends like popular versus highly rated genres, distinguish between casual and active users, and explore how individual preferences vary. Although it lacks rich metadata like descriptions or cast information, it remains highly effective for collaborative filtering and user segmentation tasks. Ultimately, it serves as a practical foundation for understanding recommendation systems and transforming user interaction data into actionable insights.

Perform exploratory analysis and visualization of user preferences

The goal of this study is to?

-Understand the genre of movie users prefer

-Assess the behaviour of users, looking at the similarities and differences

-Analyze the possibility of grouping users into patterns

Key questions to be answered:

-Which genres are most liked?

-Do heavy users behave differently?

-Can users be segmented?

Step 1: Load and Understand the Data

Here we take a quick look in to our dataset to understand, how many users and how many movies we have. We also get to see a discription of our dataset by looking at the first 5 enteries.

```
import pandas as pd

movies = pd.read_csv('/content/movies_cleaned.csv')
ratings = pd.read_csv('/content/ratings.csv')

df = ratings.merge(movies, on='movieId')

df.info()
df.describe()
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100836 entries, 0 to 100835
Data columns (total 8 columns):
 #   Column        Non-Null Count   Dtype
---  ------        --------------   -----
 0   userId        100836 non-null  int64
 1   movieId       100836 non-null  int64
 2   rating        100836 non-null  float64
 3   timestamp     100836 non-null  int64
 4   title         100836 non-null  object
 5   genres        100836 non-null  object
 6   release_year  100818 non-null  float64
 7   rating_count  100836 non-null  int64
dtypes: float64(2), int64(4), object(2)
memory usage: 6.2+ MB
```

	userId	movieId	rating	timestamp	title	genres	release_year	r
0	1	1	4.0	964982703	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy	1995.0	
1	1	3	4.0	964981247	Grumpier Old Men (1995)	Comedy|Romance	1995.0	

Step 2: Analyze Overall User Behavior

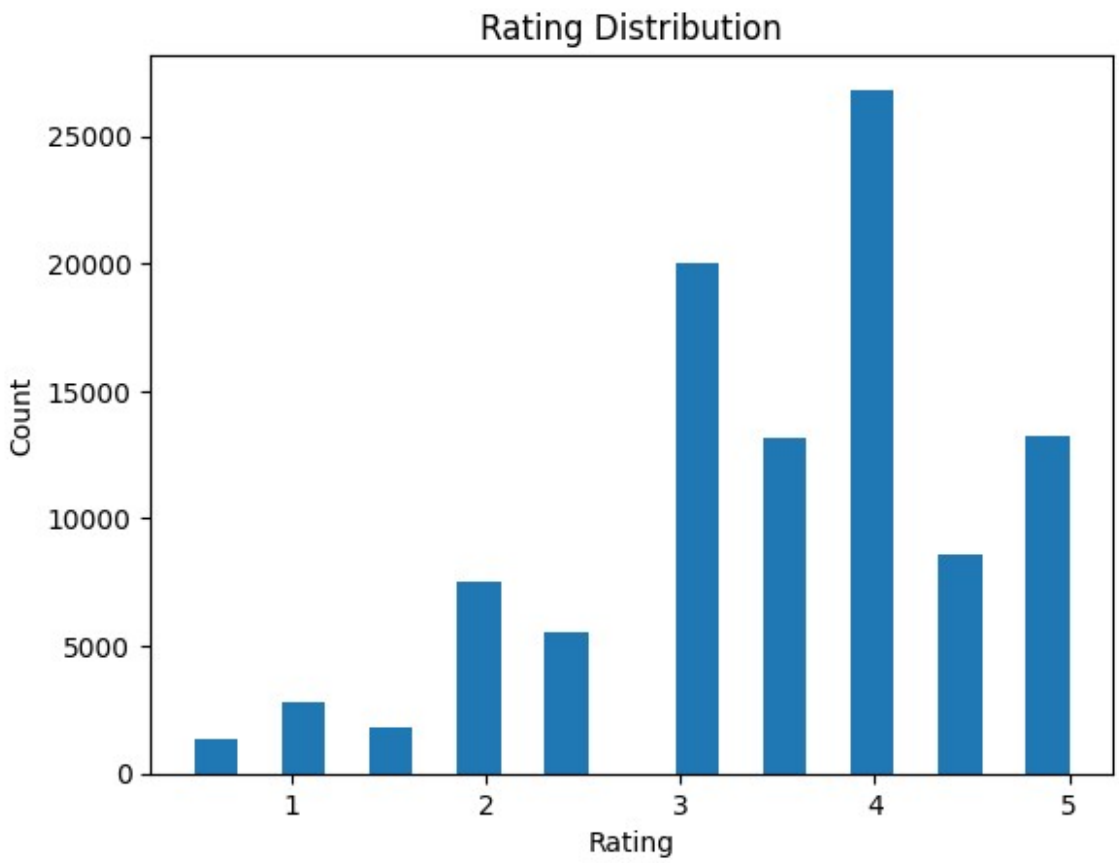
2.1 Rating Distribution

Here we will answer the following question:

-Do users give mostly high ratings?

-Is there bias in the rating?

```
import matplotlib.pyplot as plt

plt.hist(df['rating'], bins=20)
plt.title("Rating Distribution")
plt.xlabel("Rating")
plt.ylabel("Count")
plt.show()
```



Results Interpretation

This Histogram indicates that users generally give positive ratings, creating a right-skewed distribution where low ratings like 1 and 2 appear much less frequently. We observe that, the hightest rating users give for movies they watched is 4 followed by 3 and 5 respectively.

Also, we observe a positve bais (right-skewed distribution). This can be as a result of the fact that people usually rate items they like and ignore the ones they dislike.

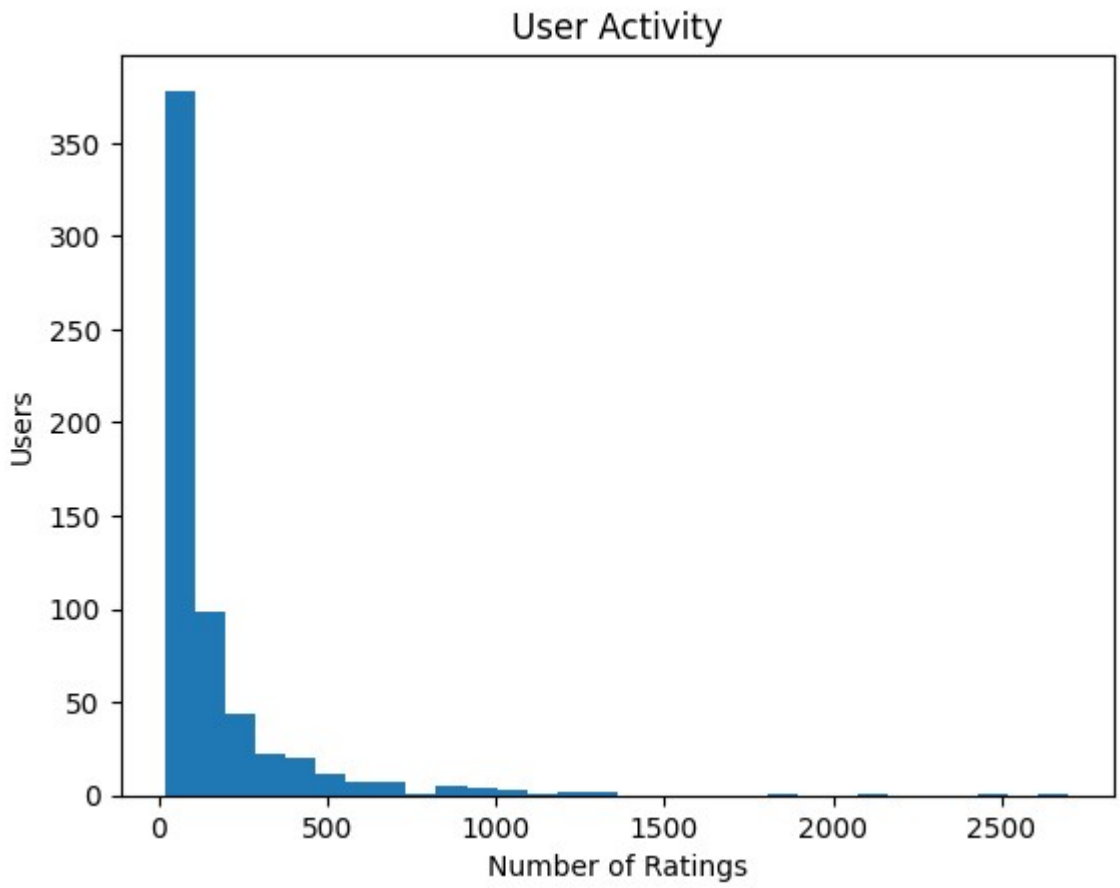
2.2 Ratings per User

Here we will answer the following question:

-How frequently users rate?

-How active our user base really is?

```
user_activity = df.groupby('userId')['rating'].count()

plt.hist(user_activity, bins=30)
plt.title("User Activity")
plt.xlabel("Number of Ratings")
plt.ylabel("Users")
plt.show()
```



Result Interpretation

-The histogram typically shows a strong imbalance, most users appear in the left side of the plot, meaning they give very few ratings. Only a small number of users extend to the right side, meaning they rate very frequently. this means, users generally rate infrequently. The majority might rate just a handful of items, while a few users rate consistently and heavily.

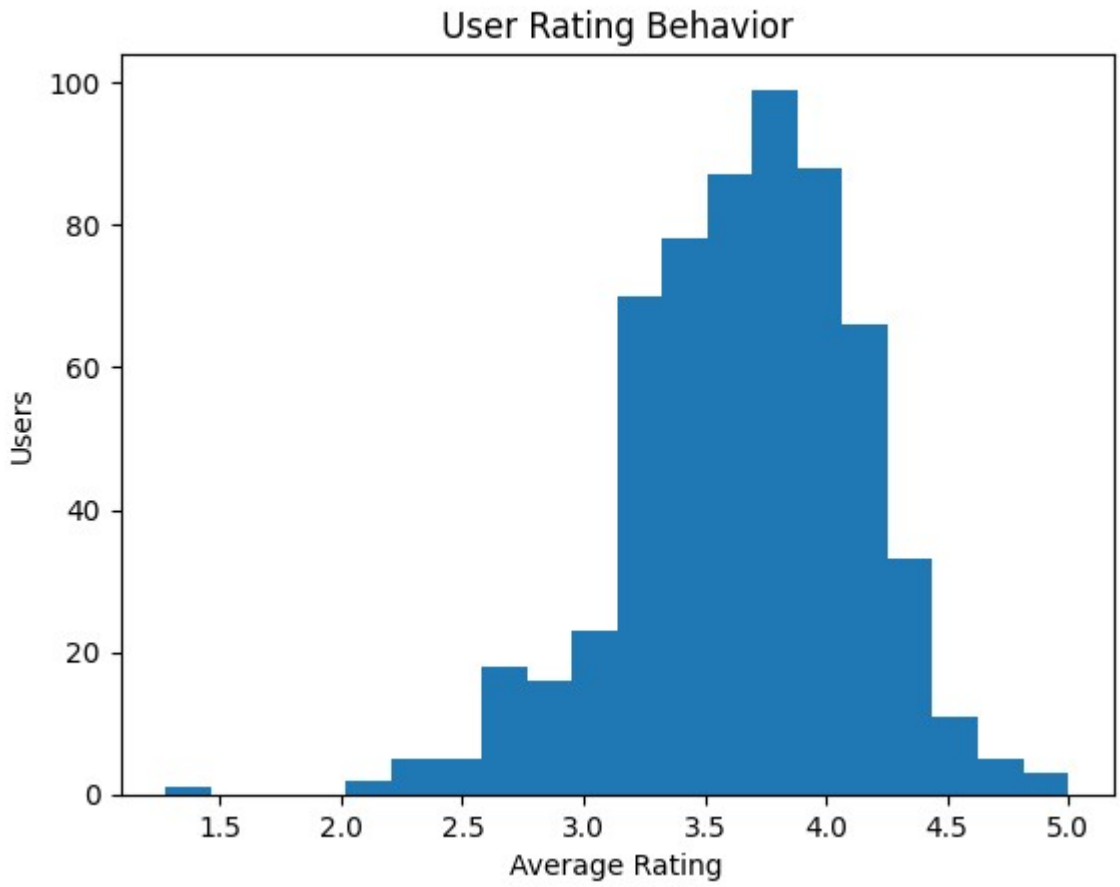
-Also, we we see that our user based is unevenly active. Most users are minimally engaged. A small group contributes a large portion of the ratings.

2.3 Average Rating per User

Here we will answer the following question:

-Who are harsh vs generous raters?

```
user_avg = df.groupby('userId')['rating'].mean()

plt.hist(user_avg, bins=20)
plt.title("User Rating Behavior")
plt.xlabel("Average Rating")
plt.ylabel("Users")
plt.show()
```



Result Interpretation

Most users are consistently giving high ratings between 3.5 and 4.0, which means they are generally lenient or positive in their scoring behavior.

Step 3: Understand Genre Preferences

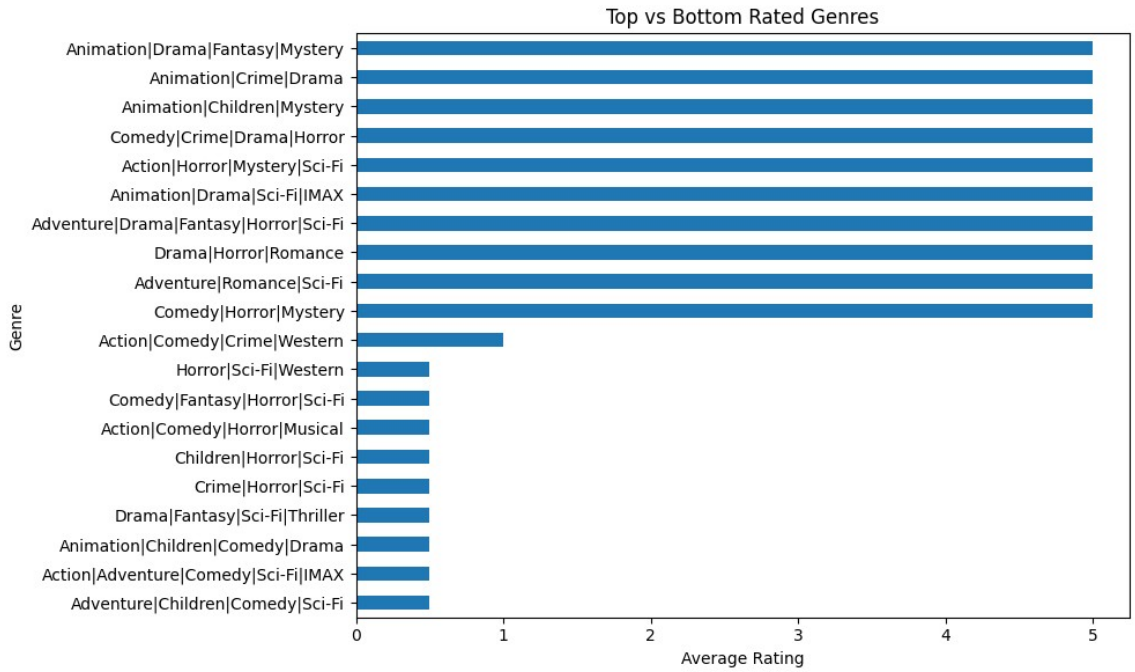
3.1 Average Rating per Genre

Here we will answer the following question:

-Which genres receive the highest and lowest average ratings from users?

```
import matplotlib.pyplot as plt

# Compute average rating per genre
```

```
genre_avg = df.groupby('genres')['rating'].mean().sort_values()

# Select top and bottom 10 genres
top_genres = genre_avg.tail(10)
bottom_genres = genre_avg.head(10)

# Combine for comparison
selected_genres = pd.concat([bottom_genres, top_genres])

# Plot
plt.figure(figsize=(10,6))
selected_genres.plot(kind='barh')

plt.title("Top vs Bottom Rated Genres")
plt.xlabel("Average Rating")
plt.ylabel("Genre")

plt.tight_layout()
plt.show()
```



Results Interpretation

-Top 10 genres (higher bars): These genres have the highest average ratings, meaning users consistently enjoy them.These are the genres users value most, not just watch.

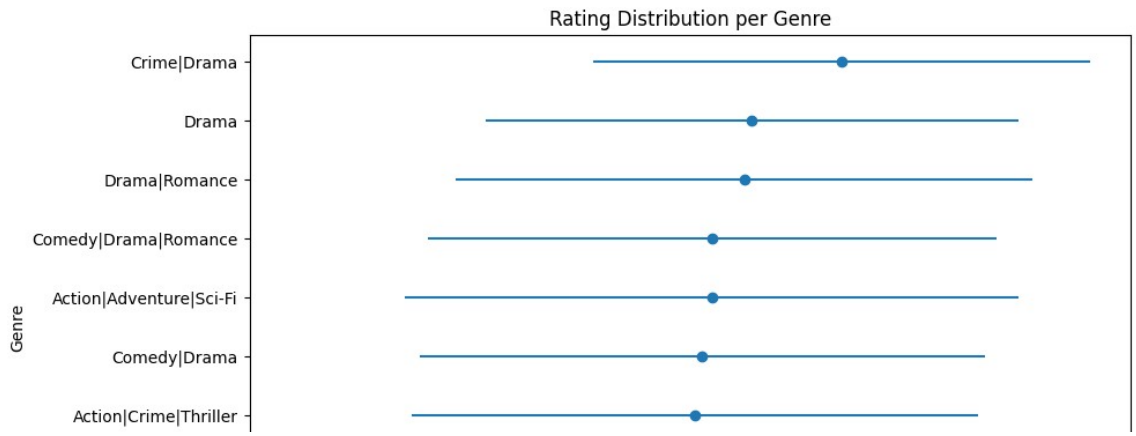
-Bottom 10 genres (lower bars): These genres have the lowest average ratings, indicating weaker user satisfaction. Users either don't enjoy these genres as much or rate them more critically.

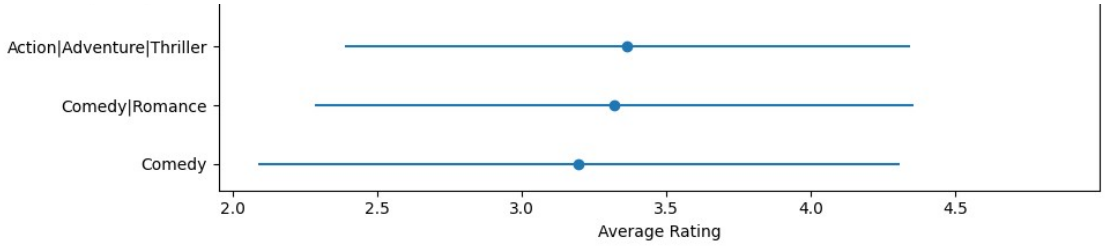
3.2 Rating Spread per Genre

Here we answer the following question:

-Which genres have higher ratings and how consistent are they

```
import matplotlib.pyplot as plt

# Top 10 genres
top_genres = df['genres'].value_counts().head(10).index
genre_sample = df[df['genres'].isin(top_genres)]

# Compute mean and standard deviation
genre_stats = genre_sample.groupby('genres')['rating'].agg(['mean', 'std'

# Plot
plt.figure(figsize=(10,6))

plt.errorbar(
    genre_stats['mean'],
    genre_stats.index,
    xerr=genre_stats['std'],
    fmt='o'
)

plt.title("Rating Distribution per Genre")
plt.xlabel("Average Rating")
plt.ylabel("Genre")

plt.tight_layout()
plt.show()
```





Result Interpretation

-Each dot represents a genre's average rating. The further right the dot, the more users generally like that genre

-The horizontal line shows variability (standard deviation) Short line → users mostly agree Long line → users have mixed opinions

*High average + short line: Users consistently rate this genre highly

*High average + long line: Some users love it, others don't

*Low average + short line: Most users agree it's not great

*Low average + long line: Mixed reactions but overall low scores

Step 4: Understanding Trends

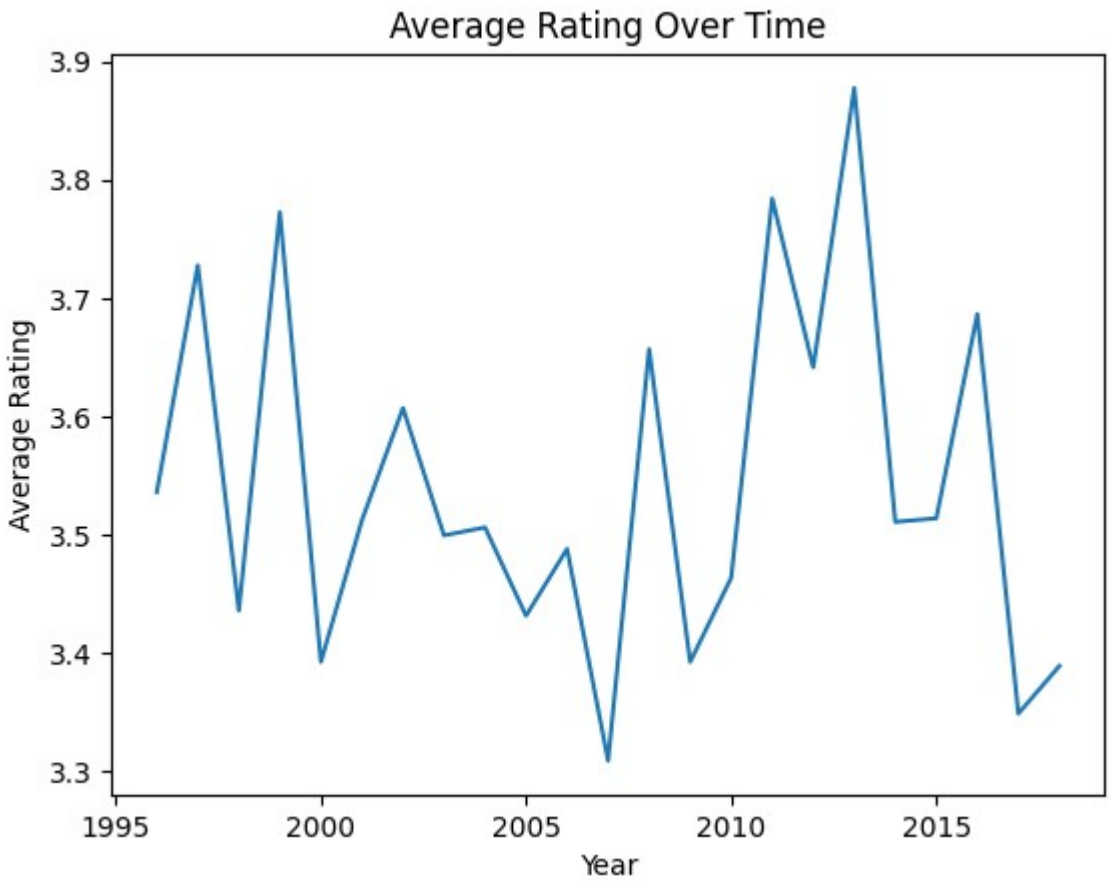
4.1 Rating Trends Over Time

Here we will answer the following question:

-Are users becoming stricter over time?

```
ratings['timestamp'] = pd.to_datetime(ratings['timestamp'], unit='s')

ratings.groupby(ratings['timestamp'].dt.year)['rating'].mean().plot()
plt.title("Average Rating Over Time")
plt.xlabel("Year")
plt.ylabel("Average Rating")
plt.show()
```

```
plt.show()
```



Result Interpretation

-When the line increases over time, it means users are giving higher ratings in recent years *Possible reasons: Better content, changing expectations, rating inflation or more satisfied with newer content.

-When the line decreases, it means users are becoming more critical over time *Possible reason: Higher expectations and more critical reviewing habits

-When the line is flat, it means user rating behavior is stable across years

Step 5. Correlation Analysis

5.1 Create Meaningful Features

Here we will answer the following question:

-How do individual users behave when rating movies

```
import pandas as pd

# Load data
movies = pd.read_csv('/content/movies_cleaned.csv')
ratings = pd.read_csv('/content/ratings.csv')
```

```
# Merge
df = ratings.merge(movies, on='movieId')

# Prepare genres
df['genres'] = df['genres'].str.split('|')
df = df.explode('genres')

# User-level features
user_features = df.groupby('userId').agg({
    'rating': ['mean', 'count', 'std'],
    'genres': 'nunique'
})

user_features.columns = ['avg_rating', 'num_ratings', 'rating_std', 'genr

user_features = user_features.dropna()
user_features.head()
```

userId	avg_rating	num_ratings	rating_std	genre_diversity
1	4.347202	697	0.807630	17
2	3.893333	75	0.754685	14
3	2.429907	107	2.073472	15
4	3.539806	515	1.324664	19
5	3.640625	128	1.040399	17

Next steps: Generate code with `user_features` New interactive sheet

Result Interpretation

-Avg_rating → average score a user gives. High score means user is lenient. Low score means user is critical

-Num_ratings → how many movies they rated. High number means power user (very engaged), Low number means casual user

-Rating_std → how consistent their ratings are. Low std means user rates consistently (predictable). High std means user is unpredictable (rates both high and low)

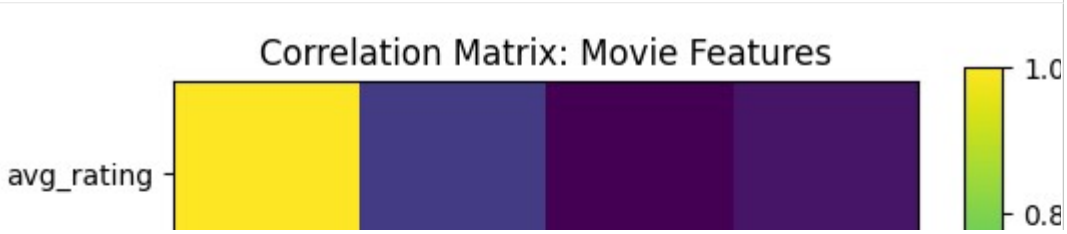
-Genre_diversity → how many different genres they watch. High value means user explores many genres. Low value means user sticks to specific genres

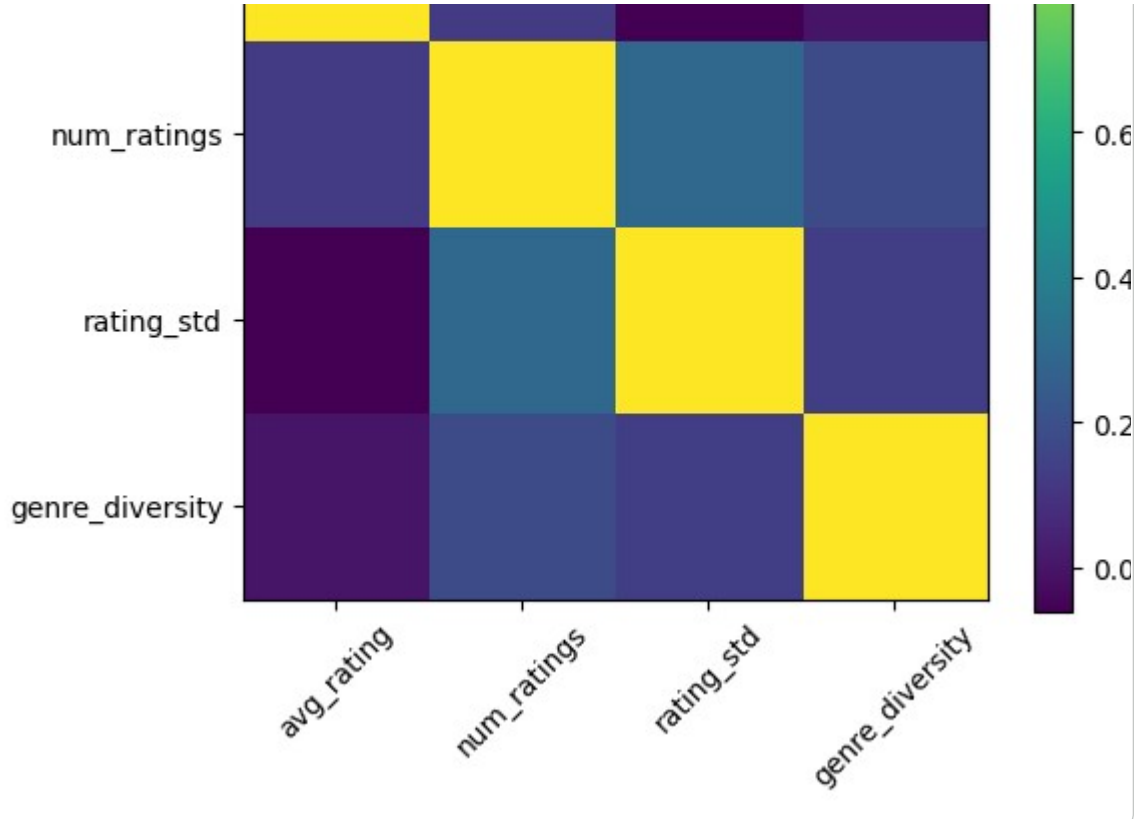
5.2 Visualize Correlation (Heatmap)

Here we will answer the following question:

-How strongly are different user behavior features related to each other?

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load datasets
movies = pd.read_csv("/content/movies_cleaned.csv")
ratings = pd.read_csv("ratings.csv")

# --- Feature Engineering ---

# Aggregate rating features
rating_stats = ratings.groupby("movieId").agg(
    avg_rating=("rating", "mean"),
    num_ratings=("rating", "count"),
    rating_std=("rating", "std")
).reset_index()

# Genre diversity (number of genres per movie)
movies["genre_diversity"] = movies["genres"].apply(
    lambda x: len(x.split("|")) if pd.notna(x) else 0
)

# Merge all features
df = movies.merge(rating_stats, on="movieId", how="left")

# Fill missing std (movies with 1 rating → std = 0)
df["rating_std"] = df["rating_std"].fillna(0)

# Drop rows with missing values in key columns
df_clean = df[["avg_rating", "num_ratings", "rating_std", "genre_diversit

# --- Correlation Matrix ---
corr_matrix = df_clean.corr()

# --- Visualization ---
plt.figure(figsize=(6,5))
plt.imshow(corr_matrix, interpolation='none')
plt.colorbar()

plt.xticks(range(len(corr_matrix.columns)), corr_matrix.columns, rotation
plt.yticks(range(len(corr_matrix.columns)), corr_matrix.columns)

plt.title("Correlation Matrix: Movie Features")
plt.show()
```





Result Interpretation

-Bright / warm colors ⟶ strong positive correlation (close to +1). Example, num_ratings vs genre_diversity

-Dark / cool colors ⟶ weak or negative correlation (toward -1). Example, avg_rating vs rating_std

-Neutral colors (around 0). Eample, avg_rating vs num_ratings

5.2 Correlation Analysis

Here we will answer the following question:

-what is the relatonship between movie release year and average rating?

```
 # Install / import libraries
import pandas as pd
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import warnings
warnings.filterwarnings("ignore")

# Run this cell in Colab to upload your CSV files manually
from google.colab import files

print("Please upload: movies_cleaned.csv, ratings.csv")
uploaded = files.upload()  # uploads go into Colab's working directory
```

```
# Load & prepare data
movies  = pd.read_csv("movies_cleaned.csv")
ratings = pd.read_csv("ratings.csv")

# Compute average rating per movie from ratings.csv
avg_ratings = (ratings.groupby("movieId")["rating"]
               .mean()
               .reset_index()
               .rename(columns={"rating": "avg_rating"}))

# Merge with movies on movieId
df = movies.merge(avg_ratings, on="movieId", how="inner")
df_clean = df[["release_year", "avg_rating"]].dropna()

print("=" * 62)
print("  CORRELATION ANALYSIS: Release Year vs Average Rating")
print("=" * 62)
print(f"\n  Total movies     : {len(df):,}")
print(f"  After NaN drop   : {len(df_clean):,}")
print(f"  Year range       : {int(df_clean['release_year'].min())} – {int
print(f"  Avg rating range : {df_clean['avg_rating'].min():.2f} – {df_cle
print(f"  Overall mean     : {df_clean['avg_rating'].mean():.3f}")

# Correlation tests
pearson_r,  pearson_p  = stats.pearsonr( df_clean["release_year"], df_cle
spearman_r, spearman_p = stats.spearmanr(df_clean["release_year"], df_cle
kendall_t,  kendall_p  = stats.kendalltau(df_clean["release_year"], df_cl

# Helper functions
def strength(r):
    a = abs(r)
    d = "positive" if r > 0 else "negative"
    s = ("strong"     if a >= 0.7 else
         "moderate"   if a >= 0.4 else
         "weak"       if a >= 0.2 else
         "negligible")
    return f"{s} {d}"

def sig(p):
    if p < 0.001: return "highly significant (p < 0.001)"
    if p < 0.01:  return "significant (p < 0.01)"
    if p < 0.05:  return "significant (p < 0.05)"
    return "NOT significant (p ≥ 0.05)"

print("\n── Correlation Results " + "─" * 39)
print(f"  Pearson  r  = {pearson_r:+.4f}  |  p = {pearson_p:.3e}  →  {str
print(f"  Spearman ρ  = {spearman_r:+.4f}  |  p = {spearman_p:.3e}  →  {s
print(f"  Kendall  τ  = {kendall_t:+.4f}  |  p = {kendall_p:.3e}  →  {str

# Per-year aggregates
yearly = (df_clean.groupby("release_year")["avg_rating"]
          .agg(mean_rating="mean", movie_count="count")
          .reset_index())
```

```
# Visualisation
fig, axes = plt.subplots(1, 2, figsize=(15, 5.5))
fig.suptitle("Correlation Analysis: Release Year vs Average Rating",
             fontsize=14, fontweight="bold", y=1.01)

# — Left: scatter + OLS regression line —
ax1 = axes[0]
ax1.scatter(df_clean["release_year"], df_clean["avg_rating"],
            alpha=0.20, s=14, color="#4C72B0")

m, b = np.polyfit(df_clean["release_year"], df_clean["avg_rating"], 1)
x_line = np.linspace(df_clean["release_year"].min(), df_clean["release_ye
ax1.plot(x_line, m * x_line + b, color="#C44E52", linewidth=2.2,
         label=f"OLS fit  (slope = {m:+.4f}/yr)")
ax1.axhline(df_clean["avg_rating"].mean(), color="grey", linestyle="--",
            linewidth=1, alpha=0.7, label=f"Grand mean ({df_clean['avg_ra

ax1.set_xlabel("Release Year", fontsize=11)
ax1.set_ylabel("Average Rating", fontsize=11)
ax1.set_title(
    f"Scatter + OLS Regression\n"
    f"Pearson r = {pearson_r:+.4f}   Spearman ρ = {spearman_r:+.4f}   p <
    fontsize=10)
ax1.set_ylim(0, 5.3)
ax1.legend(fontsize=9)
ax1.grid(axis="y", alpha=0.3)

# — Right: yearly mean rating + movie count bars —
ax2  = axes[1]
ax2b = ax2.twinx()

ax2b.bar(yearly["release_year"], yearly["movie_count"],
         color="#AECDE8", width=0.8, alpha=0.45, label="# movies")
ax2b.set_ylabel("Number of Movies", fontsize=10, color="#4C72B0")
ax2b.tick_params(axis="y", labelcolor="#4C72B0")

ax2.plot(yearly["release_year"], yearly["mean_rating"],
         color="#55A868", linewidth=1.5, alpha=0.6, label="Yearly mean ra

roll = yearly.set_index("release_year")["mean_rating"].rolling(7, center=
ax2.plot(roll.index, roll.values, color="#C44E52", linewidth=2.5, label="

ax2.set_xlabel("Release Year", fontsize=11)
ax2.set_ylabel("Average Rating", fontsize=11)
ax2.set_ylim(0, 5.3)
ax2.set_title("Yearly Mean Rating & Movie Counts", fontsize=10)
ax2.grid(axis="y", alpha=0.3)

lines1, labels1 = ax2.get_legend_handles_labels()
lines2, labels2 = ax2b.get_legend_handles_labels()
ax2.legend(lines1 + lines2, labels1 + labels2, fontsize=9, loc="lower lef

plt.tight_layout()
```

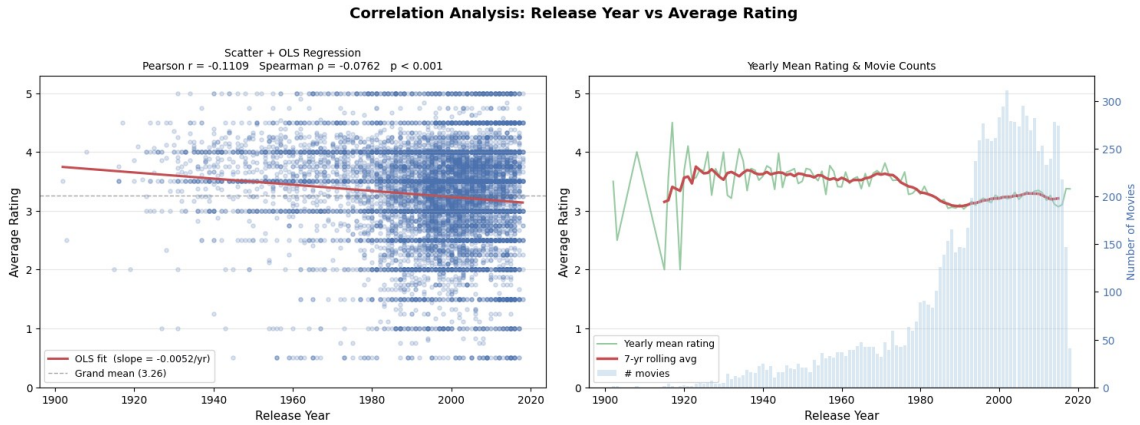
```
plt.savefig("correlation_plot.png", dpi=150, bbox_inches="tight")
plt.show()
print("Plot saved → correlation_plot.png")
```

```
Please upload: movies_cleaned.csv, ratings.csv
```

Browse... ratings.csv

movies_cleaned.csv(application/vnd.ms-excel) - 565006 bytes, last modified: n/a - 100% done

```
Saving movies_cleaned.csv to movies_cleaned (1).csv
================================================================
  CORRELATION ANALYSIS: Release Year vs Average Rating
================================================================

  Total movies      : 9,724
  After NaN drop    : 9,711
  Year range        : 1902 – 2018
  Avg rating range  : 0.50 – 5.00
  Overall mean      : 3.262

── Correlation Results ──────────────────────────────────────
  Pearson  r  = -0.1109  |  p = 5.843e-28  →  negligible negative, highly
  Spearman ρ  = -0.0762  |  p = 5.442e-14  →  negligible negative, highly
  Kendall  τ  = -0.0516  |  p = 1.242e-13  →  negligible negative, highly
```




```
Plot saved → correlation_plot.png
```

Result Interpretation

-The average ratings received by the movies reduces as the release year increases but the correlation is very low (weak correlation).

-The figure on the right shows how the average movie ratings changes over time (release year).